



KLE Technological University

Creating Value,
Leveraging Knowledge

**Department
of
Electronics and Communication Engineering**

**Minor Project Report
on
Python driven digital design using MyHdl**

By:

- | | |
|---------------------|-------------------|
| 1. Pavitra V Indi | USN: 01FE23BEC277 |
| 2. Ankit B C | USN: 01FE23BEC315 |
| 3. Srinivas Y B | USN: 01FE23BEC144 |
| 4. Shailesh S Patil | USN: 01FE23BEE102 |

**Semester: VI,
2025–2026**

**Under the Guidance of
Pooja Chandaragi**

KLE SOCIETY'S
KLE Technological University,
HUBBALLI-580031
2025-26



DEPARTMENT OF ELECTRONICS AND
COMMUNICATION ENGINEERING
CERTIFICATE

This is to certify that project entitled "Python driven digital design using MyHDL" is a bonafide work carried out by the student team of Pavitra V Indi (01FE23BEC277), Ankit B C (01FE23BEC315), Shailesh S Patil (01FE23BEE102), and Srinivas Y B (01FE23BEC144). The project report has been approved as it satisfies the requirements with respect to the Minor Project work prescribed by the university curriculum for the Bachelor of Engineering (VI Semester) in the Department of Electronics and Communication Engineering, KLE Technological University, for the academic year 2025-26.

Pooja Chandaragi
Guide

Dr. Suneeta Budihal
Head of Department

Prof. B. S. Anami
Registrar

External Viva:

Name of Examiners

1. Sangeeta V. C
2. Jyoti R. V

Signature with date

6/4/26
6/4/26

ACKNOWLEDGMENT

We would like to thank KLE Technological University, Hubballi for providing us the opportunity and support to conduct our project "Python Driven Digital Design using MyHDL. From learning environment and facilities provided by the university, we could easily complete our project. We would like to thank the Head of the School of Electronics and Communication Engineering for the encouragement and support given throughout our project. We would like to thank our Guide Prof. Pooja C for her constant guidance, clear explanations and useful suggestions, which helped us in understanding the concepts and ultimately in completing the work on time. We would like to express our gratitude to our classmates, faculty and teaching and non-teaching staff for all their support in this project.

-The project team

ABSTRACT

This project presents a Python-driven approach to digital hardware design using the MyHDL framework, where high-level Python code is used to describe, simulate, and deploy digital circuits on FPGA hardware. An 8-bit and 16-bit Booth multiplier and a Multiply-Accumulate (MAC) unit were designed in MyHDL as a datapath and FSM controller, verified through MyHDL's simulator, and then automatically converted into synthesizable Verilog code. Minor manual fixes, such as correcting output port types, tying off the unused `clrQ` signal, and restructuring FSM outputs using `always @(*)` blocks, were applied to resolve Vivado compatibility issues, after which the RTL was simulated to confirm correct FSM sequencing and accurate signed product widths. The MAC unit was wrapped as an AXI4-Lite IP core, integrated with the ZYNQ7 Processing System on the PYNQ-Z2 board, and the resulting bitstream was deployed and verified through a Jupyter Notebook using the PYNQ Python API, with hardware outputs matching expected signed-arithmetic results. The successful flow from Python-based modeling to FPGA deployment demonstrates that MyHDL provides a fast and reliable methodology for hardware design, bridging software-level design practices with digital hardware implementation.

Contents

1	Introduction	10
1.1	Motivation	11
1.2	Objectives	11
1.3	Literature Survey	12
1.3.1	Python-Based Hardware Design Frameworks	12
1.3.2	FPGA-Based Multiplier Design and Implementation	13
1.3.3	Comparative Analysis of Existing Works	14
1.3.4	Research Gap and Motivation	16
1.4	Problem statement	16
1.5	Organization of the report	17
2	System Design and Architecture	18
2.1	Proposed system overview	18
2.2	System Architecture	18
2.2.1	Hardware Design Layer	19
2.2.2	Arithmetic Processing Layer	20
2.2.3	Verification Layer	20
2.2.4	FPGA Implementation Layer	21
2.2.5	Hardware Deployment Layer	21
2.2.6	Software Interface Layer	22
2.3	Overall Working of the Proposed System	22
3	Implementation details	23
3.1	Specifications	23
3.2	Algorithm	24
3.3	Flowchart	25
3.3.1	MyHDL Design	25
3.3.2	Verilog Generation	26
3.3.3	Verilog Modifications and Testbench	26
3.3.4	Functional Simulation	26
3.3.5	Synthesis and Implementation	26

3.3.6	Bitstream Generation	26
3.3.7	PYNQ-Z2 FPGA	26
3.3.8	Python/Jupyter Verification	27
4	Results and discussions	28
4.1	Result Analysis	28
4.2	MAC Unit Functional Verification	29
4.3	Conclusion of Results	29
5	Conclusions and future scope	31
5.1	Conclusion	31
5.2	Future scope	31
5.2.1	Application in the societal context	32
	References	32
	Appendix	34

List of Tables

1.1	Comparative Analysis of Existing Literature	15
3.1	System Specifications	23

List of Figures

2.1	Overall System Architecture of the Proposed System	19
3.1	Overall Flow of the Proposed MyHDL-Based FPGA Design and Implemen- tation	25
4.1	Functional simulation of the signed MAC unit in Xilinx Vivado showing the final accumulated output.	30
4.2	Hardware validation of the signed MAC unit on the PYNQ-Z2 board using Jupyter Notebook.	30
5.1	Originality report showing a similarity index of 9%, with 8% from Internet sources and 5% from publications.	34
5.2	Originality report showing a similarity index of 9%, with 8% from Internet sources and 5% from publications.	35
5.3	Originality report showing a similarity index of 9%, with 8% from Internet sources and 5% from publications.	36

Chapter 1

Introduction

The design of digital hardware plays a crucial role in the development of modern electronic systems, supporting various applications including embedded systems, digital communication, signal processing, industrial automation and artificial intelligence. The traditional approach for creating digital circuits is to use a Hardware Description Language (HDL) like Verilog or VHDL. These languages offer full control of the implementation of the hardware, but they tend to be very long and complex to write, debug and validate, and the design process is long and difficult. With the growing complexity of digital systems, there is a need for new design methods to reduce development time with the hardware accuracy and performance. MyHDL is an open source library written in Python that allows to describe, simulate and verify digital circuits using Python. The functionality is then validated and converted to synthesizable Verilog automatically, thereby saving a lot of manual coding time. The hybrid method of combining Python with FPGA-based hardware implementation is suitable for rapid prototyping and hardware development, due to its simplicity of programming with Python and the efficiency of the FPGA implementation. The objective of this project is to propose a digital design methodology to implement an 8-bit and 16-bit Booth multiplier and a signed Multiply-Accumulate (MAC) unit in the framework of MyHDL, using a python language to implement them. The Booth Multiplier is designed to efficiently perform signed binary multiplication with minimal number of arithmetic operations, with the MAC unit allowing the accumulation of the products of successive multiplications for use in Digital Signal Processing (DSP), image processing and machine learning. The verified MyHDL designs are automatically translated to Verilog HDL, synthesized with Xilinx Vivado, and programmed on the PYNQ-Z2 FPGA board. The hardware is then checked with the use of python programs via Jupyter Notebook. This project illustrates a full design flow from Python to FPGA, emphasizing the ability of MyHDL to make hardware design easier and to produce accurate, synthesizable and hardware-ready digital circuit implementations.

1.1 Motivation

Although these traditional hardware description languages, like Verilog and VHDL, are powerful, they are also cumbersome to learn, use, and debug, particularly for students or engineers coming from a software background who usually develop faster and have less time to devote to design, debugging, and verification. Python is a popular language, easy to read and provides very powerful simulation and scripting capabilities, making it an attractive alternative for describing the behavior of hardware at a higher level of abstraction. The MyHDL framework enables hardware design to be completely modeled, simulated and verified using Python before being synthesized into Verilog, which drastically cuts design time and makes FPGA-based hardware design more accessible. The computational core of DSP and image processing hardware accelerators and AI/ML accelerators are the arithmetic units like Booth multipliers and MAC units that are perfect for showcasing this flow for Python to the hardware. Motivation of this project is to see if the Python design method is a viable way to move from rapid software-style prototyping to reliable synthesizable digital designs that are correct in the hardware.

1.2 Objectives

The main objectives of this project are:

- To learn the MyHDL framework for digital hardware design with Python.
- Create and simulate 8-bit and 16-bit Booth Multipliers using MyHDL.
- Using the signed Multiply-Accumulate (MAC) unit that is designed with MyHDL.
- To get the MyHDL designs translated into synthesizable Verilog. Run simulation and functional verification of the simulated generated Verilog in Vivado.
- To add required changes to generate a Vivado compatible synthesis of the generated Verilog.
- To design and program the FPGA (PYNQ-Z2) board to create the synthesized design.
- To connect the developed hardware with Python using Jupyter Notebook for real-time testing.
- To check correctness of signed arithmetic operations by comparing outputs from the hardware and expected results.
- To show a full design flow from Python to digital arithmetic hardware designed with MyHDL.

1.3 Literature Survey

1.3.1 Python-Based Hardware Design Frameworks

Traditional Hardware Description Languages (HDLs) such as **Verilog** and **VHDL** have been the standard design languages for implementing digital hardware systems. Although these languages provide complete control over hardware implementation, they require detailed low-level coding, making rapid prototyping, design reuse, and verification a challenging and time-consuming process. To address these limitations, researchers have explored Python-based hardware description frameworks that provide higher design abstraction while maintaining compatibility with conventional FPGA development workflows.

Among the various Python-based hardware design methodologies, **MyHDL** has emerged as one of the most widely adopted frameworks due to its ability to combine software-level abstraction with synthesizable hardware generation.

Jaic and Smith [5] enhanced the MyHDL framework by introducing attribute conversion and interface abstraction techniques. Their proposed approach significantly improves code reusability, simplifies hardware–software communication, reduces repetitive coding, and streamlines the overall FPGA development process while maintaining compatibility with synthesizable Verilog and VHDL.

Similarly, Villar *et al.* [8] demonstrated that Python can effectively function as a Hardware Description Language by comparing conventional Verilog-based development with a MyHDL-based design methodology. Their study concluded that Python offers a higher level of abstraction without compromising synthesizable hardware generation, thereby improving design productivity and reducing development complexity.

Decaluwe [2] further expanded the capabilities of MyHDL by incorporating simulation, verification, event-driven concurrency, and automatic HDL conversion within a unified framework. These features enable designers to model complex digital systems using Python while preserving deterministic hardware behaviour during synthesis, making MyHDL a powerful environment for rapid hardware prototyping.

Extending the application of Python beyond hardware description, Huang *et al.* [4] proposed a Python-based programming framework for CPU–FPGA heterogeneous systems. Their work integrates software and hardware development into a unified programming environment, abstracts low-level hardware implementation details, and simplifies the programming of heterogeneous computing platforms while enabling efficient FPGA acceleration.

Overall, the reviewed studies demonstrate that Python-based hardware description frameworks significantly improve development productivity, simplify verification, enhance code reusability, and reduce hardware design complexity while maintaining compatibility with existing FPGA synthesis tools. These advantages provide a strong foundation for implementing the proposed Booth Multiplier and Multiply-Accumulate (MAC) unit presented in this project using the MyHDL design framework.

1.3.2 FPGA-Based Multiplier Design and Implementation

Multiplication is one of the most computationally intensive arithmetic operations in digital systems and plays a fundamental role in Digital Signal Processing (DSP), image processing, cryptography, artificial intelligence, and embedded computing applications. Consequently, considerable research has focused on developing high-performance multiplier architectures that minimize propagation delay, hardware utilization, and power consumption while maximizing computational efficiency.

Among the various multiplier architectures, Array, Shift-and-Add, Booth, Modified Booth, Wallace Tree, and Vedic multipliers have been extensively investigated for FPGA implementation. Each architecture exhibits distinct trade-offs with respect to speed, logic utilization, hardware complexity, power consumption, and implementation cost. Therefore, selecting an appropriate multiplier architecture depends largely on the target application and the required performance metrics.

Anoop and Chalil [1] compared several multiplication algorithms implemented on FPGA platforms and demonstrated that the choice of multiplier architecture has a significant impact on propagation delay, hardware resource utilization, and computational efficiency. Their work emphasizes that no single multiplier is universally optimal, and the selection should be based on the specific performance requirements of the application.

Similarly, Kamble and Ugale [6] analysed Vedic, Array, Wallace Tree, and Shift-and-Add multipliers using Spartan-3 FPGA devices. Their comparative study evaluated logic

resource utilization, propagation delay, power consumption, and hardware complexity. The results indicated that Wallace Tree multipliers provide high computational speed by reducing partial products, whereas Shift-and-Add multipliers offer simpler hardware implementation with comparatively lower resource utilization.

Harika *et al.* [3] further investigated various multiplication techniques, including Carry Save Adders (CSA), Booth multipliers, Modified Booth multipliers, Wallace Tree multipliers, and Array multipliers. Their study highlighted that reducing the number of partial products and optimizing carry propagation significantly improves multiplication speed while lowering power consumption, making Booth-based multiplication highly suitable for high-performance arithmetic circuits.

A comprehensive review conducted by Nair and Saraf [7] compared Array, Modified Booth, Wallace Tree, and Modified Booth-Wallace Tree multipliers based on speed, area utilization, power consumption, and implementation complexity. Their analysis concluded that Modified Booth encoding effectively reduces the number of generated partial products, whereas Wallace Tree reduction minimizes addition stages, resulting in faster multiplication performance. Hybrid architectures combining these techniques achieve superior overall performance compared with conventional multiplier designs.

Collectively, these studies demonstrate that FPGA-based multiplier performance primarily depends on efficient partial product generation, reduction, and accumulation techniques. Among the available architectures, Booth-based multiplication provides an excellent balance between computational speed, hardware utilization, and implementation complexity, making it highly suitable for signed arithmetic operations.

Based on these observations, the present work adopts the Booth multiplication algorithm as the computational core of the proposed Multiply-Accumulate (MAC) unit. Furthermore, unlike conventional implementations that rely entirely on handwritten Verilog, the proposed design employs the MyHDL framework for high-level hardware modelling, automatic Verilog generation, manual optimization for Vivado compatibility, FPGA implementation on the PYNQ-Z2 development board, and real-time hardware validation using Jupyter Notebook.

1.3.3 Comparative Analysis of Existing Works

The reviewed literature highlights two major research directions. The first focuses on improving hardware design productivity through Python-based hardware description frameworks such as MyHDL, while the second concentrates on enhancing multiplier performance using optimized FPGA architectures. Although these studies have significantly contributed to digital hardware design, each work primarily addresses a specific aspect of the overall design flow. Table 1.1 presents a comparative summary of the reviewed literature.

Table 1.1: Comparative Analysis of Existing Literature

Reference	Technique	Major Contribution	Limitation
Jaic and Smith [5]	MyHDL Framework	Enhanced MyHDL with improved abstraction and HDL conversion	Requires manual optimization for synthesis
Villar <i>et al.</i> [8]	Python as HDL	Demonstrated Python as an effective hardware description language	Limited discussion on FPGA implementation
Decaluwe [2]	MyHDL	Provided simulation and automatic Verilog/VHDL conversion	Focused mainly on framework development
Huang <i>et al.</i> [4]	CPU-FPGA Programming	Unified Python-based software and hardware development	Targets heterogeneous computing systems
Anoop and Chalil [1]	FPGA Multipliers	Compared various multiplier architectures on FPGA	Uses conventional HDL implementation
Kamble and Ugale [6]	Vedic, Wallace, Array	Performance comparison of multiplier algorithms	No high-level hardware design methodology
Harika <i>et al.</i> [3]	Booth, Wallace	Analysed delay, power and area of multipliers	Limited implementation scope
Nair and Saraf [7]	Multiplier Review	Comprehensive comparison of multiplier architectures	Review-oriented; no hardware implementation

The comparative study indicates that existing research either focuses on high-level hardware design using Python-based frameworks or on optimizing multiplier architectures using conventional HDL techniques. Very few studies integrate both approaches into a complete FPGA design flow involving hardware modelling, automatic HDL generation, FPGA implementation, and real-time validation.

1.3.4 Research Gap and Motivation

The literature survey reveals that significant progress has been made in both Python-based hardware design frameworks and FPGA multiplier architectures. MyHDL provides a high-level design environment that simplifies hardware modelling and automatic conversion to synthesizable Verilog, while numerous studies have focused on improving multiplier performance through optimized architectures such as Booth, Wallace Tree, and Vedic multipliers.

However, most existing works address these research areas independently. Python-based frameworks primarily emphasize design abstraction and HDL generation, whereas multiplier-related research mainly concentrates on hardware optimization using manually written Verilog. Very few studies present a complete design flow that integrates high-level hardware modelling, automatic Verilog generation, manual optimization, FPGA implementation, and real-time hardware verification.

Motivated by this research gap, the present work proposes an integrated hardware design methodology for implementing a signed Booth Multiplier and Multiply-Accumulate (MAC) unit. The design is initially developed using the MyHDL framework, automatically converted into synthesizable Verilog, optimized for Vivado compatibility, implemented on the PYNQ-Z2 FPGA development board, and finally validated through Jupyter Notebook. This integrated approach combines the advantages of rapid hardware development, efficient FPGA implementation, and real-time hardware validation within a unified design flow.

1.4 Problem statement

Designing and verifying digital hardware using traditional HDLs like Verilog or VHDL is time-consuming and requires detailed low-level knowledge, slowing down rapid prototyping. Moreover, the auto-converted Verilog code from high-level descriptions may have compatibility problems with Vivado that make it impossible to synthesize or simulate correctly on this tool. This project aims to solve these problems by designing the booth multiplier and the MAC unit in Python using the MyHDL framework, converting the designs to Verilog, fixing some compatibility issues, and deploying the verified designs to a PYNQ-Z2 FPGA board.

1.5 Organization of the report

Chapter 1: Introduction

This chapter introduces the project and presents its motivation, objectives, literature survey, problem statement, and the overall organization of the report. It provides the necessary background for understanding the proposed work.

Chapter 2: System Design and Architecture

This chapter describes the overall architecture of the proposed system. It explains the MyHDL-based hardware design methodology, Booth Multiplier, MAC unit, Verilog generation, FPGA implementation, and hardware verification process.

Chapter 3: Implementation Details

This chapter discusses the implementation of the proposed system. It includes the system specifications, architecture, algorithm, flowchart, and explains each stage from MyHDL design to FPGA deployment and verification.

Chapter 4: Results and Discussion

This chapter presents the simulation and hardware implementation results of the Booth Multiplier and MAC unit. It also analyzes the obtained outputs and verifies the correctness of the proposed design.

Chapter 5: Conclusion and Future Scope

This chapter summarizes the work carried out during the project and highlights the major achievements. It also discusses possible future enhancements and the societal applications of the proposed system.

Chapter 2

System Design and Architecture

2.1 Proposed system overview

The proposed system is designed in a digital design flow, implemented in Python and designed with MyHDL framework. The hardware modules are designed and verified in Python, synthesized in Verilog and then implemented on the PYNQ-Z2 FPGA in Vivado. The system is composed of Booth Multiplier (8-bit and 16-bit), Multiply-Accumulate (MAC) unit and Python interface for testing and verification.

2.2 System Architecture

The proposed system adopts an integrated Python-driven hardware design methodology for implementing a signed Booth Multiplier and Multiply-Accumulate (MAC) unit on the PYNQ-Z2 FPGA platform. Unlike conventional FPGA development, where hardware modules are manually described using Verilog or VHDL, the proposed approach utilizes the MyHDL framework to model digital hardware using Python. The hardware design is functionally verified through simulation, automatically converted into synthesizable Verilog, synthesized using the Xilinx Vivado Design Suite, and deployed onto the PYNQ-Z2 FPGA board for real-time hardware execution. The overall system architecture consists of six functional layers, namely the Hardware Design Layer, Arithmetic Processing Layer, Verification Layer, FPGA Implementation Layer, Hardware Deployment Layer, and Software Interface Layer. Each layer performs a dedicated function in the complete hardware development flow while ensuring seamless interaction between the software and FPGA hardware components.

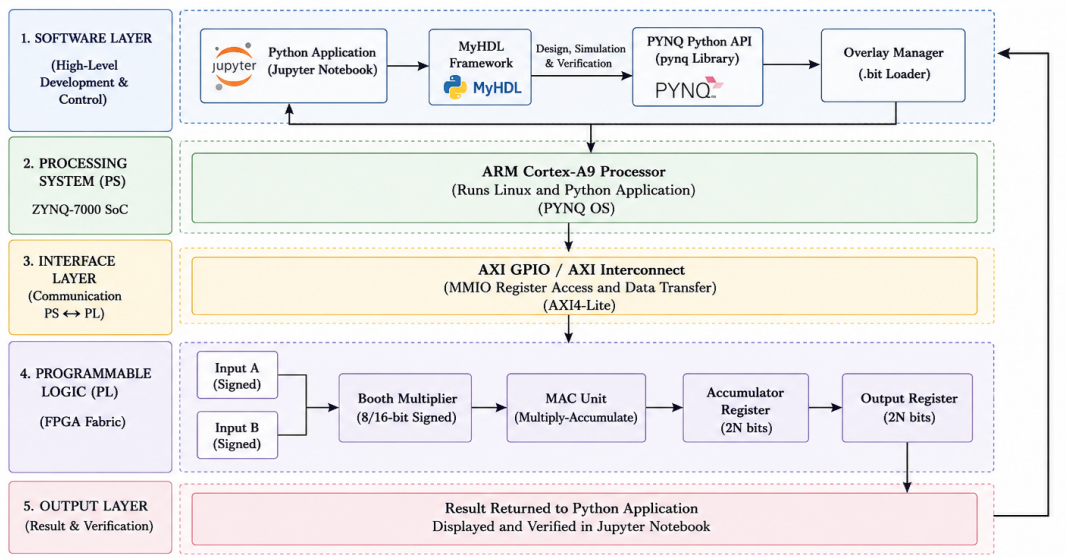


Figure 2.1: Overall System Architecture of the Proposed System

The proposed system architecture consists of five functional layers, namely the Software Layer, Processing System (PS), Interface Layer, Programmable Logic (PL), and Output Layer. These layers collectively perform hardware modelling, FPGA implementation, and software-based verification of the proposed Booth Multiplier and MAC unit. The functionality of each layer is described in the following subsections.

2.2.1 Hardware Design Layer

The Hardware Design Layer serves as the foundation of the proposed system architecture. In this layer, the digital hardware modules are developed using the MyHDL framework, which enables hardware circuits to be described using Python syntax while preserving synthesizable hardware behaviour. Compared with conventional Hardware Description Languages (HDLs) such as Verilog and VHDL, MyHDL provides a higher level of abstraction, making hardware modelling, debugging, and functional verification more efficient.

The proposed system consists of an 8-bit Booth Multiplier, a 16-bit Booth Multiplier, and a signed Multiply-Accumulate (MAC) unit. Each hardware module is initially designed and functionally verified within the MyHDL environment before being translated into synthesizable Verilog. This design methodology allows rapid prototyping and simplifies modifications during the early stages of hardware development.

Using MyHDL also enables automatic HDL generation from a single Python-based description, thereby reducing manual coding effort and minimizing implementation errors. The generated Verilog forms the basis for subsequent FPGA synthesis and deployment on the PYNQ-Z2 platform.

2.2.2 Arithmetic Processing Layer

The Arithmetic Processing Layer performs the core computational operations of the proposed system. It consists of the Booth Multiplier, Multiply-Accumulate (MAC) unit, Accumulator Register, and Output Register, all of which are implemented within the Programmable Logic (PL) section of the PYNQ-Z2 FPGA. These hardware modules work together to execute signed multiplication and accumulation operations efficiently.

The Booth Multiplier receives two signed input operands and performs multiplication using Booth's algorithm. Two versions of the multiplier, namely 8-bit and 16-bit, are implemented to demonstrate scalability and support different operand sizes. Booth's algorithm minimizes the number of arithmetic operations by reducing partial products, thereby improving computational efficiency and reducing hardware complexity.

The multiplication result generated by the Booth Multiplier is supplied directly to the MAC unit. The MAC unit performs multiplication followed by accumulation in a single hardware module by adding each newly generated product to the previously stored accumulated value. This operation is widely used in Digital Signal Processing (DSP), image processing, machine learning, and convolution-based applications, where repeated multiply-accumulate operations are required.

The accumulated result is stored in the Accumulator Register before being transferred to the Output Register. The Output Register provides a stable output interface for subsequent hardware verification through the Processing System (PS) and the Python application running in Jupyter Notebook. This layered arithmetic architecture enables efficient execution while maintaining modularity and ease of integration with the remaining FPGA design.

2.2.3 Verification Layer

The Verification Layer ensures the functional correctness of the proposed hardware design before FPGA implementation. After developing the Booth Multiplier and MAC unit in the MyHDL framework, the hardware modules are simulated using the MyHDL simulation environment. This stage verifies that the designed hardware behaves as expected under different input conditions.

A dedicated testbench is developed to apply various combinations of signed input operands to both the 8-bit and 16-bit Booth Multipliers as well as the MAC unit. The generated outputs are compared with the expected arithmetic results to verify the correctness of multiplication and accumulation operations. Any functional errors identified during simulation are corrected before proceeding to hardware synthesis.

Functional verification at this stage significantly reduces debugging effort during FPGA implementation. Since the design is thoroughly validated before HDL generation, the probability of synthesis and implementation errors is minimized, resulting in a reliable

and efficient hardware development process.

2.2.4 FPGA Implementation Layer

The FPGA Implementation Layer converts the verified hardware design into a form that can be programmed onto the PYNQ-Z2 FPGA board. Once the functionality of the Booth Multiplier and MAC unit is verified through simulation, the MyHDL framework automatically translates the Python-based hardware description into synthesizable Verilog HDL.

The generated Verilog design is imported into the Xilinx Vivado Design Suite for synthesis and implementation. During this stage, the hardware design is analyzed, optimized, and mapped onto the FPGA resources. Minor modifications are incorporated into the generated Verilog wherever necessary to ensure compatibility with the Vivado synthesis tool and to eliminate synthesis warnings.

After successful synthesis and implementation, Vivado generates a configuration bitstream (.bit) file. This bitstream contains the complete hardware configuration required to program the Programmable Logic (PL) section of the PYNQ-Z2 FPGA. The generated bitstream serves as the hardware image that is loaded onto the FPGA during deployment.

2.2.5 Hardware Deployment Layer

The Hardware Deployment Layer is responsible for programming the synthesized hardware design onto the PYNQ-Z2 FPGA development board. After successful generation of the configuration bitstream in Vivado, the bitstream is transferred to the FPGA through the PYNQ environment. The programmable logic (PL) of the Zynq-7000 device is configured with the generated hardware design, making the Booth Multiplier and MAC unit available for real-time execution.

The PYNQ-Z2 platform integrates an ARM Cortex-A9 Processing System (PS) with FPGA Programmable Logic (PL), enabling efficient communication between software and hardware. Once the overlay is loaded, the Processing System initializes the hardware modules, manages data transfer, and controls the execution of arithmetic operations. This hardware–software co-design approach provides high computational efficiency while maintaining the flexibility of Python-based application development.

The deployed hardware can be repeatedly programmed and tested without modifying the physical circuitry, making the PYNQ-Z2 platform highly suitable for rapid prototyping, hardware validation, and embedded system development.

2.2.6 Software Interface Layer

The Software Interface Layer provides communication between the Python application and the hardware implemented on the PYNQ-Z2 FPGA platform. After programming the FPGA with the generated bitstream, the Python application running in Jupyter Notebook loads the hardware overlay using the PYNQ Python library. This enables software control of the Booth Multiplier and MAC unit without requiring any additional hardware programming.

Communication between the Processing System (PS) and the Programmable Logic (PL) is achieved through the AXI GPIO interface. The Python application writes the signed input operands to the FPGA registers, initiates the arithmetic operation, and reads the computed output from the output register. This communication mechanism allows seamless data transfer between software and hardware while maintaining efficient execution.

The hardware-generated results are displayed in the Jupyter Notebook and compared with the expected software-computed values to verify the correctness of the implemented design. This real-time verification process demonstrates successful integration of Python-based hardware control with FPGA acceleration, providing an efficient platform for rapid hardware development, testing, and validation.

2.3 Overall Working of the Proposed System

The overall operation of the proposed system begins with modelling the Booth Multiplier and MAC unit using the MyHDL framework in Python. The hardware modules are functionally verified through simulation and automatically translated into synthesizable Verilog. The generated Verilog is synthesized and implemented using the Xilinx Vivado Design Suite, which produces the FPGA configuration bitstream. The bitstream is then programmed onto the PYNQ-Z2 FPGA board, enabling execution of the arithmetic hardware modules. Finally, Python programs executed through Jupyter Notebook communicate with the FPGA using the PYNQ library and AXI GPIO interface to provide input operands, control hardware execution, retrieve the computed results, and verify the correctness of the implemented design. Figure 3.1 illustrates the complete workflow of the proposed system.

Chapter 3

Implementation details

3.1 Specifications

The specifications of the proposed Python-driven digital design system are summarized in Table 3.1.

Table 3.1: System Specifications

Parameter	Specification
Project Title	Python-Driven Digital Design using MyHDL
Design Framework	MyHDL (Python)
Programming Language	Python 3
Hardware Description Language	Verilog HDL (Generated from MyHDL)
Arithmetic Modules	8-bit Booth Multiplier, 16-bit Booth Multiplier, Signed MAC Unit
FPGA Design Tool	Xilinx Vivado
Target FPGA Board	PYNQ-Z2 (Zynq-7000 SoC)
Verification Environment	Jupyter Notebook (Python)
Synthesis Target	Xilinx Zynq-7000 FPGA
Supported Arithmetic	Signed Multiplication and Multiply-Accumulate (MAC)
Output	Correct signed multiplication and accumulated results

3.2 Algorithm

Booth Multiplication Algorithm

The Booth multiplication algorithm is a signed binary multiplication technique that reduces the number of addition and subtraction operations by examining pairs of bits in the multiplier. The algorithm operates as follows:

Step 1: Load the multiplicand into register M and the multiplier into register Q. Clear the accumulator register A and the extra bit qm1 to zero. Set the iteration counter to n, where n is the bit width of the operands (8 or 16).

Step 2: Examine the current least significant bit of Q (q0) and the previous bit qm1 as a pair.

Step 3: If the bit pair is 01, add M to A and store the result back in A.

Step 4: If the bit pair is 10, subtract M from A and store the result back in A.

Step 5: If the bit pair is 00 or 11, no addition or subtraction is performed.

Step 6: Perform an arithmetic right shift on the combined register pair {A, Q} by one bit, shifting qm1 in from the left of A and Q[0] becoming the new qm1.

Step 7: Decrement the counter by one.

Step 8: Repeat steps 2 through 7 until the counter reaches zero.

Step 9: The final 2n-bit signed product is read from the combined {A, Q} register pair.

FSM Controller Algorithm

The FSM controller sequences through the following states to drive the datapath:

Step 1: IDLE state — the FSM waits for the start signal to be asserted.

Step 2: INIT state — on receiving start, the controller clears A and qm1, loads M with the multiplicand, and loads the counter with n iterations.

Step 3: LOAD state — the controller loads Q with the multiplier value.

Step 4: TEST state — the controller checks whether the counter has reached zero. If yes, it transitions to the DONE state. If no, it examines the {q0, qm1} bit pair and transitions to the appropriate ADD, SUB, or SHIFT state.

Step 5: ADD state — the controller asserts ldA and addsub=1, causing A to be updated with $A + M$.

Step 6: SUB state — the controller asserts ldA and addsub=0, causing A to be updated with $A - M$.

Step 7: SHIFT state — the controller asserts sftA, sftQ, and decr, performing the arithmetic right shift on {A, Q} and decrementing the counter, then returns to the TEST state.

Step 8: DONE state — the controller asserts the done signal and holds it, making the final product available on the {A_out, Q_out} output ports.

MAC Unit Algorithm

Step 1: Clear the accumulator register to zero before starting a new dot product or sequence of operations.

Step 2: Accept two operands A and B as inputs.

Step 3: Invoke the Booth multiplier with A as the multiplicand and B as the multiplier and wait for the done signal.

Step 4: Read the product from the Booth multiplier output.

Step 5: Add the product to the current value stored in the accumulator register.

Step 6: Store the updated sum back into the accumulator register.

Step 7: Repeat steps 2 through 6 for all input operand pairs.

Step 8: The final value in the accumulator register is the result of the multiply-accumulate operation, representing the dot product or running sum as required.

3.3 Flowchart

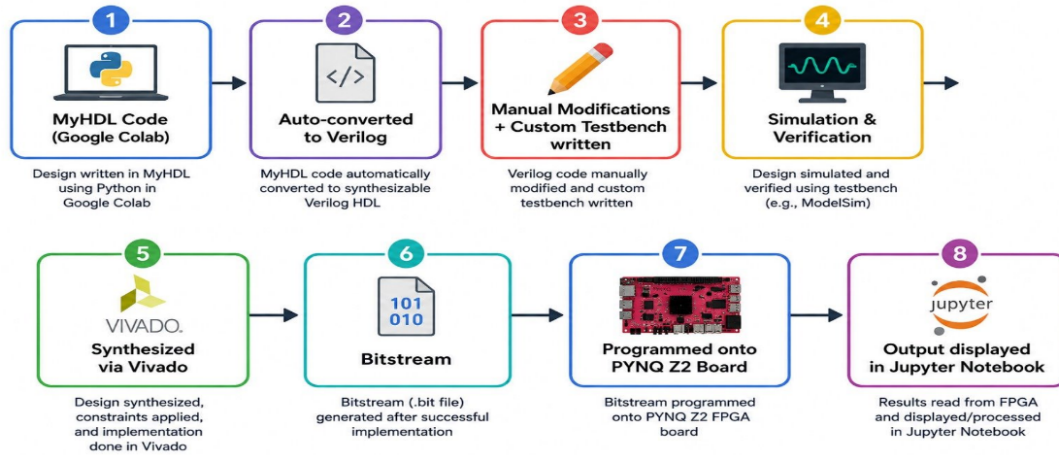


Figure 3.1: Overall Flow of the Proposed MyHDL-Based FPGA Design and Implementation

3.3.1 MyHDL Design

The design process begins with the MyHDL framework, where the Booth Multiplier (8-bit and 16-bit) and the MAC unit are developed using Python. MyHDL allows digital hardware to be described using Python syntax, making the design process simpler and easier to debug compared to traditional HDL-based development. Functional verification is also carried out at this stage before proceeding to hardware implementation.

3.3.2 Verilog Generation

After the MyHDL design is verified, the hardware description is automatically converted into synthesizable Verilog HDL. This eliminates the need to manually rewrite the design in Verilog and reduces coding effort. The generated Verilog serves as the hardware description used for FPGA implementation.

3.3.3 Verilog Modifications and Testbench

The automatically generated Verilog code is reviewed and modified wherever necessary to ensure compatibility with the Xilinx Vivado synthesis tool. A custom Verilog testbench is also developed to provide input test vectors and verify the functionality of the Booth Multiplier and MAC unit before synthesis.

3.3.4 Functional Simulation

The modified Verilog design is simulated using the testbench to verify its functional correctness. During this stage, different combinations of signed input values are applied to ensure that the Booth multiplication algorithm, FSM controller, and MAC unit operate correctly. Simulation helps identify and eliminate design errors before hardware implementation.

3.3.5 Synthesis and Implementation

After successful simulation, the Verilog design is imported into the Xilinx Vivado Design Suite. The synthesis process converts the HDL description into FPGA hardware resources such as lookup tables, flip-flops, and registers. The implementation stage performs placement and routing, followed by the generation of the FPGA configuration bitstream.

3.3.6 Bitstream Generation

Once synthesis and implementation are completed successfully, Vivado generates a bitstream (.bit) file. This file contains the complete hardware configuration required to program the FPGA. The bitstream acts as the final hardware image that will be loaded onto the PYNQ-Z2 board.

3.3.7 PYNQ-Z2 FPGA

The generated bitstream is programmed onto the PYNQ-Z2 FPGA board. The Booth Multiplier and MAC unit are implemented as dedicated hardware modules within the programmable logic of the FPGA. Hardware execution provides faster and more efficient arithmetic computation than software-based execution.

3.3.8 Python/Jupyter Verification

In the final stage, Python programs running in Jupyter Notebook communicate with the FPGA using the PYNQ framework. The application loads the hardware overlay, sends input operands to the FPGA, initiates the arithmetic operation, retrieves the computed outputs, and compares them with the expected software results. This confirms the correctness of the implemented hardware and completes the Python-to-FPGA design workflow.

Chapter 4

Results and discussions

4.1 Result Analysis

A Python based digital design flow has been successfully realized by the MyHDL framework. The following circuits were designed, simulated, synthesized into Verilog, and implemented on the PYNQ-Z2 FPGA using Xilinx Vivado: an 8-bit Booth Multiplier, a 16-bit Booth Multiplier, and a signed Multiply-Accumulate (MAC) unit.

The functional simulation verified that both Booth Multiplier designs produced correct signed multiplication results for different combinations of positive and negative operands. The generated Verilog code was compatible with Xilinx Vivado after minor modifications and was synthesized successfully without any functional errors.

The MAC unit performed multiply-accumulate operations by multiplying the input operands and adding the product to the accumulator register. The accumulator retained the cumulative sum of successive multiplication operations, and the obtained outputs matched the expected arithmetic results.

The generated FPGA bitstream was deployed on the PYNQ-Z2 board, and Python programs were executed through the Jupyter Notebook environment to provide input operands to the hardware IP and read back the computed outputs. The hardware results were compared with software-calculated values, and both matched exactly, validating the correctness of the complete Python-to-FPGA design flow.

Overall, the implementation demonstrates that MyHDL provides an efficient hardware design methodology capable of generating synthesizable Verilog for rapid FPGA deployment. The successful implementation of the Booth Multiplier architectures and the MAC unit validates the proposed design approach for signed arithmetic operations in digital signal processing and related embedded hardware applications.

4.2 MAC Unit Functional Verification

Figure 4.1 shows the functional simulation of the signed MAC unit performed in Xilinx Vivado. The simulation verifies the correct operation of the finite state machine, Booth multiplier, and accumulator.

Initially, the MAC unit remains in the `MAC_IDLE` state until the start signal is asserted. During execution, the Booth multiplier computes the signed multiplication of the input operands while the accumulator stores the cumulative result. After the completion of all multiply-accumulate operations, the MAC unit returns to the idle state and asserts the completion flag.

For the test sequence applied during simulation, the accumulator reached a final value of **89**, which agrees with the expected software calculation. The waveform and simulation log confirm that the multiplier, accumulator, and control logic operate correctly without overflow or functional mismatch.

The successful simulation confirms the correctness of the proposed MAC architecture generated through the MyHDL design flow. The obtained waveform validates the state transitions, Booth multiplication process, accumulator update, and completion signal generation, thereby verifying the functionality of the complete hardware design before FPGA implementation.

4.3 Conclusion of Results

The proposed Python-based hardware design methodology successfully bridges high-level hardware modeling with FPGA implementation. The generated Verilog modules were synthesized and implemented on the PYNQ-Z2 FPGA without functional issues. Both Booth Multiplier architectures produced correct signed multiplication results, while the MAC unit accurately performed cumulative multiply-accumulate operations. Simulation, synthesis, and FPGA validation demonstrated complete agreement between software and hardware outputs, confirming the correctness and reliability of the design. These results establish that MyHDL is an effective framework for rapid FPGA-based digital hardware development and is well suited for implementing arithmetic building blocks used in DSP, machine learning accelerators, and embedded computing applications.

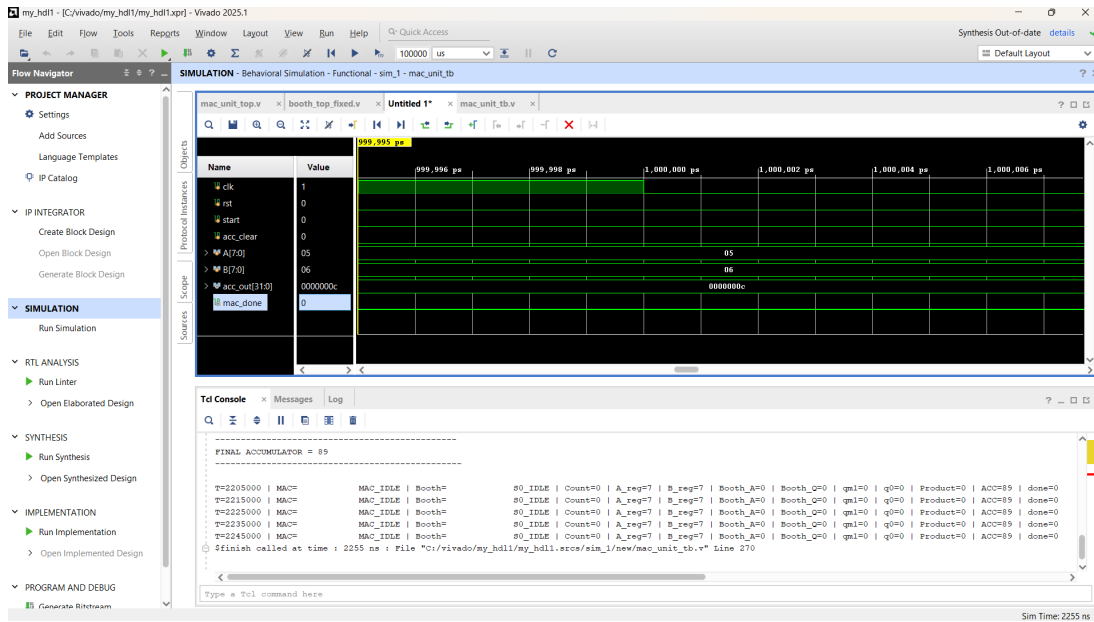


Figure 4.1: Functional simulation of the signed MAC unit in Xilinx Vivado showing the final accumulated output.

```
In [2]: from pynq import Overlay

ol = Overlay("myhdl_mac.bit") # auto-loads the matching mac_booth.huh
print("IPs found:", list(ol.ip_dict.keys()))
IPs found: ['mac_unit_axi_0', 'processing_system7_0']

In [3]: mac = ol.mac_unit_axi_0
print(mac) # shows the IP object + its base address (0x40000000)
<pynq.overlay.DefaultIP object at 0xb38b26e8>

In [5]: def s8(x):
    """Pack a signed int into 8-bit two's complement (-1 -> 0xFF)."""
    return x & 0xFF

def s32(v):
    """Interpret a 32-bit register read as signed."""
    return v - (1 << 32) if (v & 0x80000000) else v

In [6]: def mac_op(a, b):
    # slv_reg0 layout: [7:0]-A, [15:8]-B, [16]-start, [17]-acc_clear
    word = s8(a) | (s8(b) << 8) | (1 << 16) # operands + start=1
    mac.write(0x00, word) # assert start
    mac.write(0x00, word & ~(1 << 16)) # deassert -> re-arm the edge detector
    while (mac.read(0x08) & 1) == 0: # poll sticky-done at offset 0x08
        pass
    return s32(mac.read(0x04)) # read accumulator at offset 0x04

In [7]: # Clear the accumulator first (acc_clear = bit 17)
mac.write(0x00, (1 << 17))
mac.write(0x00, 0)

print(mac_op(3, 4)) # expect 12
print(mac_op(5, 6)) # expect 42 (12 + 30)
print(mac_op(-1, 2)) # expect 40 (42 - 2)
print(mac_op(7, 7)) # expect 89 (40 + 49)

12
42
40
89
```

Figure 4.2: Hardware validation of the signed MAC unit on the PYNQ-Z2 board using Jupyter Notebook.

Chapter 5

Conclusions and future scope

5.1 Conclusion

This project was a successful example of a complete digital design flow from high level hardware modeling to real FPGA deployment on the PYNQ-Z2 board, all implemented in a Python program. An 8-bit and 16-bit Booth multiplier and a Multiply-Accumulate (MAC) unit have been designed and simulated all in Python and automatically translated to synthesizable Verilog. The output port types in the automatically converted code had to be fixed, the undriven signal `clrQ` had to be “tied off” and the combinational output logic of the FSM had to be restructured with the help of `always @(*)` blocks to avoid latch inference in Vivado. These changes were successfully simulated, synthesized and implemented on the Zynq-7000 SoC. The MAC unit was embedded as an AXI4-Lite IP core into a PYNQ-Z2 block design and tested using a Jupyter Notebook via the PYNQ Python API, generating hardware outputs which matched the results obtained from signed arithmetic. The project concludes that MyHDL is a quick, accurate and widely applicable approach to digital hardware design and is able to close the gap between Python abstraction level and synthesizable, hardware correct FPGA implementation.

5.2 Future scope

This proposed work can be expanded on by many facets to enhance performance and to enable more advanced digital hardware applications.

- Implement higher bit Booth Multipliers (e.g., 32-bit, 64-bit).
- Design pipelined and parallel MAC architectures for high speed computation.
- Use several MACs to execute vector and matrix operations.
- Extend the design for Digital Signal Processing (DSP) applications such as FIR filters and FFT processors.

- Leverage the AI, ML accelerators, including CNN calculations, in the hardware.
- Discuss optimization methods for minimizing FPGA resources and increasing the operating frequency.
- Do the entire MyHDL-to-FPGA design flow automatically, without manual changes to the generated Verilog.

5.2.1 Application in the societal context

The proposed hardware design methodology using Python could be applicable in various domains which has immediate impact on the society. Booth Multipliers and MAC units are key components of digital signal processing, communication, medical electronics, artificial intelligence, robotics and embedded systems. By using MyHDL, hardware development can be easier, quicker, and faster to prototype, and less time is needed to develop FPGA based solutions. This makes development of affordable and dependable digital systems for medical devices, smart transportation, industrial automation, image processing, and edge AI applications easier. The proposed methodology is also expected to foster learning, research and innovation in modern digital system design by making it more accessible through Python hardware design.

Bibliography

- [1] Anoop C. and Anu Chalil. Implementation and analysis of multiplication algorithms for vlsi applications using fpga. *International Journal of Electrical, Electronics and Data Communication*, 5(9):48–51, 2017.
- [2] Jan Decaluwe. Myhdl: Designing digital hardware with python, 2010. Presentation and Tutorial.
- [3] K. Harika, B. V. Swetha, B. Renuka, D. Lakshman Rao, and S. Sridhar. Analysis of different multiplication algorithms and fpga implementation. *IOSR Journal of VLSI and Signal Processing*, 4(2):29–35, 2014.
- [4] Sitao Huang, Kun Wu, Sai Rahul Chalamalasetti, Izzat El Hajj, Cong Xu, Paolo Faraboschi, and Deming Chen. A python-based high-level programming flow for cpu-fpga heterogeneous systems. In *IEEE/ACM Programming Environments for Heterogeneous Computing (PEHC)*, pages 20–26, 2021.
- [5] Keerthan Jaic and Melissa C. Smith. Enhancing hardware design flows with myhdl. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 28–31, 2015.
- [6] Manoj M. Kamble and Sunita P. Ugale. Fpga implementation and analysis of different multiplication algorithm. *International Journal of Computer Applications*, 149(2):33–36, 2016.
- [7] Savita Nair and Ajit Saraf. A review paper on comparison of multipliers based on performance parameters. In *International Conference on Advances in Science and Technology (ICAST)*, pages 6–9, 2014.
- [8] J. I. Villar, J. Juan, M. J. Bellido, J. Viejo, D. Guerrero, and Jan Decaluwe. Python as a hardware description language: A case study. In *IEEE Conference*, pages 117–122, 2011.



Figure 5.1: Originality report showing a similarity index of 9%, with 8% from Internet sources and 5% from publications.

14	escholarship.org	Internet Source
15	mme.fiu.edu	Internet Source
16	O. Slimani, K. Boudjit. "Acceleration and Energy Optimization of Convolutional Neural Networks on Zynq-7000 SoC: Comparative Analysis with GPU Platforms", Journal Européen des Systèmes Automatisés, 2026	Publication
17	files.core.ac.uk	Internet Source
18	repositori.uji.es	Internet Source
19	dspace.daffodilvarsity.edu.bd:8080	Internet Source
20	wiredspace.wits.ac.za	Internet Source
21	Mary D. Pulukuri, George J. Starr, Charles E. Stroud. "On Built-In Self-Test for multipliers Proceedings of the IEEE SoutheastCon 2010 (SoutheastCon), 2010	Publication
22	pure.tue.nl	Internet Source
23	www.slideshare.net	Internet Source
24	dspace.mit.edu	Internet Source
25	tabbforum.com	Internet Source
26	ijirt.org	Internet Source
27	open.uct.ac.za	Internet Source
28	Devansh Chawda, Benaoumeur Senouci. "Fast prototyping of Quantized neural network on an FPGA edge computing device with Brevitas and FINN", 2024 Fifteenth International Conference on Ubiquitous and Future Networks (ICUFN), 2024	Publication

Figure 5.2: Originality report showing a similarity index of 9%, with 8% from Internet sources and 5% from publications.

29	Dhirendra Kumar Shukla, Shabir Ali, Sandhya Sharma. "Artificial Intelligence and Sustainable Innovation - Volume 2", CRC Press, 2026	Publication
30	lutpub.lut.fi	Internet Source
31	sitaohuang.com	Internet Source

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		

Figure 5.3: Originality report showing a similarity index of 9%, with 8% from Internet sources and 5% from publications.